

SUMMER PROJECT REPORT INTERNSHIP

Project Title

Student Academic Performance Prediction

Submitted By:

- **Student Name:** PEETHA YASWANTH
- **Course & Batch:** B.Sc. B.Ed. in Mathematics with Minor in Computer Science
- **Institute Name:** NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL

Period of Internship: 18th May 2026 – 14th June 2026

Report submitted to: IDEAS – Institute of Data Engineering, Analytics and Science Foundation, ISI Kolkata

1. Abstract

This project focuses on building a machine learning model to predict student academic outcomes using a dataset of 3,000 student records containing 32 features. The data includes various metrics tracking student demographics, study habits, psychological states, and academic scores. During the project, data preprocessing steps were performed, which included identifying categorical and numerical features, handling missing values, encoding text labels, and scaling the data. A Decision Tree Classifier was trained on the processed data to categorize student performance. The model achieved a perfect testing accuracy of 100%. Further analysis revealed that the performance categories were directly determined by a single feature: the final exam score. The results show how tree-based models can quickly isolate exact underlying business rules in structured datasets.

2. Introduction

Predicting student performance is highly relevant in modern education because it allows institutions to identify students who might need extra help early in the semester. This project uses machine learning tools to look at how different behavioral, environmental, and academic factors relate to a student's final performance category. "The core technology involved is Python", along with data science libraries like Pandas for data manipulation and Scikit-Learn for preprocessing and building the predictive model. The procedure used follows standard data science practices: loading raw data, cleaning missing records, transforming variables, splitting the data for valid testing, and training a classifier. The main purpose of doing this project is to apply theoretical machine learning steps to a structured, multifeature dataset to see how well computational models can map student habits to academic success.

Training Received: -

During the first two weeks of my foundational course these are the list of topics that I received training

- Basic Statistics for Data Science
- Data Visualization
- Introduction to GitHub and Cloud Computing
- Data Science Application Development using Streamlit

3.Project Objective

The main goals of this project are:

- To clean and preprocess a multi-feature academic dataset by handling missing entries and normalization.
- To separate independent student features from the target evaluation column.
- To split the data correctly into training and testing sets to prevent model overfitting.
- To train a Decision Tree Classifier to predict the student performance category.
- To evaluate the model's accuracy and analyze feature importances to see which student habits matter the most.

4.Methodology

The workflow for this project is written in a simple step by step manner:

1. **Data Loading:** The student_performance_dataset.csv file containing 3,000 rows and 32 columns was loaded using Pandas.
2. **Column Identification:** Columns were separated by data type, finding 26 numerical columns (like study hours and attendance) and 6 categorical text columns (like gender and career goals).
3. **Data Imputation:** The dataset had exactly 60 missing rows each in social_media_hours, focus_score, and sleep_hours. A loop was written to fill the continuous columns with their median values and text columns with their mode, bringing remaining missing values to zero.
4. **Label Encoding:** LabelEncoder was used to turn text columns into numerical integers so the math models could read them.
5. **Data Split:** The data was split into features (**X**) and target (**y**). It was partitioned into 80% for training (2,400 rows) and 20% for testing (600 rows) using a fixed random state of 42.
6. **Feature Scaling:** MinMaxScaler was applied, to scale all input features strictly between 0.0 and 1.0 so that larger raw numbers wouldn't bias the model.
7. **Model Selection and Validation:** A Decision Tree Classifier was chosen for this classification task. It was validated by training it strictly on the training set and making it predict values on the hidden test set to calculate the final accuracy score.

5.Data Analysis and Results

Descriptive Summary of Missing Data

Before cleaning the data, the initial missing value scan showed the following counts:

Column Name	Missing Values (Before)	Missing Values (After Imputation)	Imputation Method
social_media_hours	60	0	Column Median
focus_score	60	0	Column Median
sleep_hours	60	0	Column Median
All other 29 columns	0	0	None Required

Model Performance

After training, the Decision Tree model achieved the following performance on the test set:

- Measured Prediction Accuracy: 1.0 (100% accuracy) Feature Importance Results

To understand the model's perfect score, the importance weights were pulled directly from the tree model:

Feature Name	Importance Score
final_exam_score	1.0
age	0.0
study_hours_per_day	0.0
All other 29 features	0.0

Question 4 output confirming 0 remaining missing values.

Question 4: Data Imputation (3 Marks)

Fill missing numerical values with the median and missing categorical values with the mode. Print the total number of missing values remaining in the entire dataset.

```
[ ] 1 for col in numerical_cols:
    df[col].fillna(df[col].median(), inplace=True)

    for col in categorical_cols:
        df[col].fillna(df[col].mode()[0], inplace=True)

    print("Remaining Missing Values:")
    print(df.isnull().sum().sum())

2 ... Remaining Missing Values:
0
/tmp/ipykernel_1291/1345587153.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the

    df[col].fillna(df[col].median(), inplace=True)
/tmp/ipykernel_1291/1345587153.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the

    df[col].fillna(df[col].mode()[0], inplace=True)
```

Question 9 output showing 1.0 model accuracy on the test set.

Question 9: Decision Tree Classifier (5 Marks)

Initialize and train a `DecisionTreeClassifier` with `random_state=42`. Make predictions on the test set and print the accuracy score.

Expected Output:

A numerical value representing the accuracy score.

```
[ ] 1 from sklearn.tree import DecisionTreeClassifier
    from sklearn.metrics import accuracy_score

    model = DecisionTreeClassifier(random_state=42)

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    print("Accuracy:", accuracy)

2 Accuracy: 1.0
```

Question 10 output showing `final_exam_score` importance at 1.0.

```
[ ] 1 importance = pd.Series(
    model.feature_importances_,
    index=X.columns
)

    print(importance.sort_values(ascending=False))

2 final_exam_score      1.0
   age                  0.0
   student_id           0.0
   city_type            0.0
   study_hours_per_day  0.0
   deep_work_sessions   0.0
   gender               0.0
   attendance_percentage 0.0
   social_media_hours   0.0
   doomscrolling_before_sleep 0.0
   notification_distractions 0.0
   ai_tool_usage_hours  0.0
   gaming_hours         0.0
   stress_level          0.0
   assignment_completion_rate 0.0
   motivation_level     0.0
   focus_score           0.0
   mental_state          0.0
   procrastination_index 0.0
   caffeine_intake       0.0
   physical_activity_hours 0.0
   internet_quality      0.0
   sleep_hours           0.0
   family_support        0.0
   financial_stress      0.0
   career_goal           0.0
   learning_style        0.0
   productivity_after_midnight 0.0
   revision_efficiency   0.0
```

6.Conclusion

The project successfully implemented a machine learning pipeline to analyze student data. The Decision Tree Classifier achieved a perfect accuracy of 1.0 because the feature importance analysis showed that the target `performance_category` is entirely derived from a single metric: `final_exam_score`. The model discovered that all other behavioral and environmental features had an

importance of 0.0 because the performance label is just a direct, rule-based categorization of the exam grade. For future work, it is recommended to drop the final exam score feature entirely from the dataset to see if the model can use behavioral habits (like study hours and attendance) to predict performance when the exact exam score is unknown.

7.APPENDICES

Appendix A: References

- Kaggle Dataset Source: [Academic Performance Prediction Dataset](#)
- Pandas Documentation: <https://pandas.pydata.org/>
- Scikit-Learn Ensemble and Tree Models: <https://scikit-learn.org/stable/modules/tree.html>

Appendix B: GitHub Link

- **Project Code Repository:** <https://github.com/yaswanthp0879-pyc/Student-Academic-Performance-Prediction>
- **Appendix C: Document Storage**
- A copy of this project notebook file (11_Student_Academic_Performance_Prediction_Summer_2026.ipynb) and this final report are securely backed up and hosted within the main branch of the GitHub repository listed in Appendix B.